

Encoder vs LLM for Intent & Entity Understanding in Domain-Specific Conversational AI

Problem Statement

Build a domain-specific conversational understanding system that identifies user intent and extracts relevant entities from natural language queries. Develop two parallel solutions — a custom lightweight Encoder-based model (BERT-style) and a modern LLM-based approach (prompting or few-shot) — and perform a rigorous comparative evaluation across performance, efficiency, and deployment-readiness.

Dataset Requirements

Select a specific domain (e.g., banking, e-commerce, healthcare, travel). Construct a custom annotated dataset with:

- 5–10 distinct user intents and 10–20 entity types (BIO-tagged)
- Minimum 800–1000 annotated utterances with paraphrastic diversity and realistic noise
- Standard train/val/test split with stratification by intent class

Module 1: Data Preparation & Feature Engineering

Task 1: Dataset Construction & Annotation (1.5 Marks)

Design and build a domain-specific intent-entity dataset. Define the intent taxonomy and entity schema with clear boundary-case examples. Generate diverse utterances per intent including paraphrases, informal phrasing, and ambiguous queries. Annotate entities using BIO tagging. Perform exploratory data analysis covering intent distribution, entity frequency, and vocabulary coverage.

Task 2: Text Preprocessing & Tokenization (1.5 Marks)

Prepare the corpus for both modeling paradigms:

- Encoder path: Train a subword tokenizer (WordPiece/BPE) on the domain corpus. Show quantitative evidence that domain-specific tokenization reduces OOV rates vs. a generic tokenizer.
- LLM path: Design structured prompt templates encoding the intent taxonomy and entity schema. Experiment with at least two prompt formats (instruction-based vs. in-context examples).
- Handle BIO tag alignment with subword token boundaries explicitly.

Module 2: Model Development

Task 3: Encoder-Based Model for Intent & Entity Recognition (2 Marks)

Build a lightweight Encoder-only Transformer (BERT-style) that jointly performs intent classification and entity extraction. Implementation must include:

- Multi-Head Self-Attention, Feed-Forward Networks, Layer Normalization, Residual Connections
- Training with AdamW, learning rate scheduling, and early stopping

Task 4: LLM-Based Intent & Entity Extraction (1.5 Marks)

Implement an LLM-based solution using an accessible generative model. Students must:

- Design at least three prompting strategies: (a) zero-shot, (b) few-shot, (c) chain-of-thought or structured output (JSON-mode)
- Implement robust output parsing with graceful handling of malformed responses
- Report per-query inference latency and token consumption per strategy
- Discuss failure modes: hallucinated entities, misclassified intents, inconsistent formats

Module 3: Comparative Evaluation & Analysis

Task 5: Quantitative Benchmarking (2 Marks)

Conduct a head-to-head evaluation covering:

- Intent Metrics: Accuracy, Precision, Recall, F1 (macro & weighted). Present confusion matrix.
- Entity Metrics: Entity-level F1 (strict & partial match). Per-entity-type breakdown.
- System Metrics: Inference latency (ms/query), throughput, model size, cost per 1000 queries.
- Robustness: Test on at least 20 adversarial/OOD cases (misspellings, ambiguous queries, unseen entity values).

Task 6: Qualitative Analysis & Deployment Recommendation (1.5 Marks)

Provide a comprehensive analytical discussion:

- Comparative analysis table: Encoder vs LLM across all dimensions
- At least 10 error analysis examples (5 per model) with diagnosis
- Trade-off analysis: accuracy vs. efficiency, cost vs. scalability, generalization vs. controllability

- Deployment recommendation: under what constraints should each paradigm be preferred?

Note: Students must provide comprehensive explanations and inferences for every task. Your documentation should clearly articulate why Encoder-only architectures remain relevant for language-understanding tasks despite the emergence of large generative models, and under what conditions each paradigm offers a superior trade-off.

Deliverables for the Assignment

Students must submit:

1. A complete Python notebook (Ipython Notebook Only)
2. PDF of the ipynb notebook with Results
3. Custom annotated dataset files (with annotation schema documentation)
4. Encoder model architecture implementation from scratch
5. LLM prompting pipeline with prompt templates
6. Training loop with loss calculation (for the Encoder model)
7. Comparative evaluation results with visualizations
8. Output examples with analysis
9. A short technical report explaining design decisions, challenges, and limitations

All files must be zipped and uploaded as a single .zip file when submitting the assignment.